

Общая схема пайплайна рендеринга (итоговая итерация 4)

В проекте реализован **гибридный рендеринг** с элементами отложенного рендеринга (через G-буфер не используется, но есть промежуточный буфер для постобработки) и прямого рендеринга. Основной упор сделан на физически корректное освещение (PBR) с окружением (IBL). Пайплайн выглядит следующим образом:

```
text
1. Инициализация приложения
  └─ Создание окна (Win32)
  └─ Инициализация DirectX 11 (устройство, контекст, цепочка обмена)
  └─ Создание ресурсов:
      └─ Буферы глубины и цвета (с учётом reversed depth)
      └─ Промежуточный буфер цвета для постпроцессинга
      └─ Загрузка текстур (альбедо, нормали, детали, flow, roughness)
      └─ Генерация геометрии ландшафта (из EXR-карты высот)
      └─ Инициализация IBL (пошаговая генерация cubemap, irradiance, prefiltered, BRDF LUT)
      └─ Создание скайбокса (сфера с кубической текстурой)
      └─ Создание константных буферов, состояний растеризатора/глубины/смешивания, сэмплеров
  └─ Инициализация ImGui (UI)

2. Главный цикл рендеринга (Render)
  └─ Обновление времени (deltaTime)
  └─ Обновление IBL (по одному шагу за кадр, если не готово)
  └─ Обновление камеры (UpdateCamera)
      └─ Обработка ввода с клавиатуры (WASD, QE)
      └─ Поворот камеры (мышь)
      └─ Подстройка высоты над рельефом (интерполяция по карте высот)
      └─ Формирование матриц вида и проекции (reversed depth)
  └─ Заполнение и обновление SceneBuffer (матрица VP, позиция камеры, источники света, параметры материалов, режимы)
  └─ Обновление PostProcessBuffer (выбранный эффект)

  └─ Рендеринг в промежуточный цветовой буфер (m_pColorBufferRTV)
      └─ Очистка цветового буфера и буфера глубины (reversed depth: 1.0)
      └─ Отрисовка скайбокса (Skybox)
          └─ Установка DepthStencilState (только тест, без записи)
          └─ Установка RasterizerState (CULL_FRONT)
          └─ Использование кубической карты (из IBL) как текстуры
          └─ Рисование сферы (индексированный треугольник)
      └─ Отрисовка ландшафта (Terrain)
          └─ Установка DepthStencilState (запись глубины, reversed depth)
          └─ Установка текстур (альбедо, нормали, детали, flow, roughness)
          └─ Установка IBL-текстур (irradiance, prefiltered, BRDF LUT)
          └─ Установка константных буферов (геометрический + сцены)
          └─ Применение PBR-шейдера (TerrainPS.hlsl)
              └─ Вычисление параметров материала (roughness, metalness, F0)
              └─ Flow-эффект (модификация albedo, roughness, F0, occlusion, specular IBL)
              └─ Точечное освещение (PBR для каждого источника)
              └─ Направленный свет (солнце/луна) - используется направление из IBL
              └─ IBL диффузный (irradiance map) + зеркальный (prefiltered + BRDF LUT)
              └─ Применение occlusion и specular-occlusion от Flow
              └─ Отладочные режимы (рендеринг отдельных компонент PBR/IBL)
              └─ Тональное отображение (Reinhard) + гамма-коррекция
          └─ Рисование индексированной геометрии ландшафта
      └─ (Опционально: рендеринг маленьких сфер-лампочек для источников света)

  └─ Применение постпроцессинга (RenderPostProcess)
      └─ Переключение цели рендера на задний буфер (m_pBackBufferRTV)
      └─ Установка полноэкранного вершинного шейдера (рисует треугольник)
      └─ Установка пиксельного шейдера постпроцессинга
```

	— Передача промежуточного цветового буфера как текстуры (SRV) — Применение выбранного эффекта (сепия, холодный тон, ночное зрение) — Вывод результата в задний буфер
	- Рендеринг ImGui (поверх всего) — ImGui_ImplDX11_NewFrame / ImGui_ImplWin32_NewFrame — Создание панели управления (настройки освещения, PBR, режимы, постобработка) — Отрисовка ImGui через ImGui_ImplDX11_RenderDrawData — Презентация цепочки обмена (SwapChain->Present)

Общая таблица файлов проекта (сводка)

Файл	Назначение	Основные функции/структуры	Соответствие лекциям (раздел)
DirectX11App.cpp	Точка входа, инициализация окна, DirectX, основных ресурсов, главный цикл рендеринга, обработка ввода, управление камерой.	WinMain, InitWindow, InitDirectX, InitTerrain, InitShaders, InitBuffers, LoadTexture (и др. загрузчики), InitSmallSpheres, InitPostProcess, Render, UpdateCamera, ResizeSwapChain, Cleanup, обработчики клавиш/мыши.	Лекция 1 (диагностика), Лекция 2 (HDR/тонирование), Лекция 3 (PBR), Лекция 8 (тени – не реализованы, но упоминаются)
DirectX11App.h	Заголовочный файл приложения (содержит только #include "resource.h").	—	—
framework.h	Стандартные заголовки Windows и C-рантайма.	—	—
targetver.h	Определение версии Windows SDK.	—	—
Resource.h	Идентификаторы ресурсов (меню, иконки, диалоги).	IDS_APP_TITLE, IDR_MAINFRAME и др.	—
Light.h	Структура источника света и функция расчёта освещения по модели Фонга (для визуализации маленьких сфер).	struct Light, CalculateColor (фонговская функция для отладки).	Лекция 3 (PBR – но используется только для сфер-лампочек)
Texture.h / Texture.cpp	Загрузка текстур из DDS (BC-сжатие) и PNG (через WIC) с генерацией MIP-цепочки.	LoadDDS, LoadPNG, GetBytesPerBlock, DivUp, WCSToMBS.	Лекция 6 (3D модели) – упоминание текстур, но не привязано к конкретной лекции.
IBL.h / IBL.cpp	Генерация Image-Based Lighting ресурсов: кубическая карта из HDR, irradiance map, prefilered env map, BRDF LUT. Пошаговое выполнение (по кадрам).	class IBL: Init, Update, Shutdown, RenderCubemapFace, RenderIrradianceFace, RenderPrefilterStep, RenderBRDF, FindSunDirectionFromHDR.	Лекция 4 (IBL diffuse), Лекция 5 (IBL specular).
PBR.hlsl	Общие функции для PBR-модели Кука-Торранса (NDF, Geometry, Fresnel) и вспомогательные (GetF0).	D_GGX, G_SchlickGGX, G_Smith, F_Schlick, FresnelSchlickRoughness, GetF0.	Лекция 3 (PBR).
TerrainVS.hlsl	Вершинный шейдер ландшафта. Принимает позицию, касательную, нормаль, UV; преобразует в мировое пространство и передаёт параметры материала (roughness, metalness, наличие карты нормалей) в пиксельный шейдер.	main (VS), использует GeomBuffer (матрица модели) и SceneBuffer (матрица VP).	Лекция 3 (PBR) и общие принципы вершинных шейдеров.
TerrainPS.hlsl	Пиксельный шейдер ландшафта. Реализует полный PBR+IBL, Flow-эффект, отладочные режимы, тональное отображение и гамма-коррекцию.	main (PS), использует текстуры (альbedo, нормали, детали, flow, roughness), IBL-текстуры (irradiance, prefilered, BRDF LUT), константные буферы с параметрами сцены и света.	Лекция 3 (PBR), Лекция 4 (IBL diffuse), Лекция 5 (IBL specular).
SkyboxVS.hlsl	Вершинный шейдер скайбокса. Передаёт направление на кубическую карту, убирает перенос из матрицы вида.	main (VS), использует константный буфер с матрицами view/proj.	Лекция 4 (IBL – скайбокс как часть окружения).
SkyboxPS.hlsl	Пиксельный шейдер скайбокса. Сэмплирует кубическую текстуру (из IBL) с учётом ночного затухания.	main (PS), использует skyboxTexture (регистр t9) и skyboxSampler.	Лекция 4.
HDRToCubeMap_VS.hlsl	Вершинный шейдер для рендеринга кубической карты из HDR-панорамы. Рисует полноэкранный квад, вычисляет направление в мировом пространстве из обратной матрицы view-projection.	main (VS), использует invViewProj.	Лекция 4 (преобразование equirectangular → cubemap).

Файл	Назначение	Основные функции/структуры	Соответствие лекциям (раздел)
HDRToCubeMap_PS.hlsl	Пиксельный шейдер для преобразования HDR-панорамы в кубическую карту. Сэмплирует 2D-текстуру по сферическим координатам.	main (PS), использует tex (регистр t0).	Лекция 4.
IrradianceMap_PS.hlsl	Пиксельный шейдер для генерации irradiance map (диффузный IBL). Интегрирует по полусфере с заданным числом семплов.	main (PS), использует кубическую текстуру tex, константный буфер params (Nphi, Ntheta).	Лекция 4.
PrefilterEnvMap_PS.hlsl	Пиксельный шейдер для генерации prefiltered environment map (спекулярный IBL). Применяет importance sampling GGX для разных roughness.	main (PS), использует кубическую текстуру tex, константный буфер roughness4.	Лекция 5.
BRDFLUT_PS.hlsl	Пиксельный шейдер для генерации 2D-текстуры BRDF LUT (scale и bias). Вычисляет интеграл по BRDF для всех комбинаций NdotV и roughness.	main (PS), IntegrateBRDF, ImportanceSampleGGX, geometry (Smith).	Лекция 5.
FullscreenQuad_VS.hlsl	Вершинный шейдер для полноэкранного треугольника (используется при генерации BRDF LUT, irradiance, prefiltered).	main (VS), использует SV_VertexID.	Лекция 4, 5.
compile_shaders.bat	Пакетный файл для компиляции HLSL-шейдеров в CSO-файлы с помощью fxc.exe.	—	—
Презентация к отчёту.pdf	Описание итераций разработки, визуальные результаты, заключение.	—	—

Это общая структура.

Детальный разбор файлов `DirectX11App.cpp` И `DirectX11App.h`

1. Описание файла `DirectX11App.h`

Этот файл является минимальным заголовочным файлом приложения. Он содержит только одну директиву:

```
cpp

#pragma once
#include "resource.h"
```

Назначение: обеспечивает включение идентификаторов ресурсов (меню, иконок, диалогов), которые определены в `Resource.h`. Сам по себе не содержит никакой логики, лишь связывает ресурсы с проектом.

2. Описание файла `DirectX11App.cpp`

Это главный файл приложения, содержащий точку входа, инициализацию всех компонентов DirectX, загрузку ресурсов, главный цикл рендеринга, обработку ввода и управление камерой. Код насчитывает более 1000 строк, поэтому я разобью его на логические блоки.

2.1. Глобальные переменные и константы (внешние и статические)

Переменная / Константа	Тип	Назначение	Связь с лекциями
<code>g_hWnd</code>	<code>HWND</code>	Дескриптор главного окна.	—
<code>m_pDevice, m_pDeviceContext</code>	<code>ID3D11Device*</code> , <code>ID3D11DeviceContext*</code>	Устройство и контекст DirectX 11.	Лекция 1 (диагностика) — используются для создания ресурсов и отрисовки.
<code>m_pSwapChain</code>	<code>IDXGISwapChain*</code>	Цепочка обмена для вывода на экран.	—
<code>m_pBackBufferRTV</code>	<code>ID3D11RenderTargetView*</code>	Представление заднего буфера как цели рендера.	—
<code>g_ibl</code>	<code>IBL</code>	Глобальный экземпляр класса для генерации IBL-ресурсов.	Лекции 4, 5 (IBL).
<code>m_lights[10]</code>	<code>Light</code>	Массив точечных источников света (до 10).	Лекция 3 (PBR).
<code>m_ambientColor</code>	<code>XMFLOAT4</code>	Цвет фонового (ambient) освещения.	—
<code>m_lightCount</code>	<code>int</code>	Количество активных источников.	—
<code>m_useNormalMaps</code>	<code>bool</code>	Флаг использования карт нормалей.	—
<code>m_showNormals</code>	<code>bool</code>	Флаг визуализации нормалей (отладка).	—

Переменная / Константа	Тип	Назначение	Связь с лекциями
m_showLightBulbs	bool	Показывать ли сферы-лампочки для источников света.	–
m_pTextureNM, m_pTextureViewNM	ID3D11Texture2D*, ID3D11ShaderResourceView*	Текстура карты нормалей и её SRV.	–
m_pDetailTextureView	ID3D11ShaderResourceView*	Текстура деталей (detail map).	–
m_pFlowTextureView	ID3D11ShaderResourceView*	Текстура Flow (для эффекта потоков).	–
m_flowModeIndex	int	Индекс режима Flow (0-3).	–
m_pRoughnessTextureView	ID3D11ShaderResourceView*	Текстура шероховатости (roughness).	–
m_pMetalnessTextureView	ID3D11ShaderResourceView*	Текстура металличности (пока не используется).	–
g_renderMode	int	Режим отладки рендеринга (0 – полный PBR+IBL, 1-10 – отдельные компоненты).	–
m_pColorBuffer, m_pColorBufferRTV, m_pColorBufferSRV	ID3D11Texture2D*, ID3D11RenderTargetView*, ID3D11ShaderResourceView*	Промежуточный буфер цвета для постпроцессинга.	Лекция 2 (HDR/постобработка).
m_pPostProcessVertexShader, m_pPostProcessPixelShader	ID3D11VertexShader*, ID3D11PixelShader*	Шейдеры постпроцессинга (полноэкранный треугольник).	–
m_pPostProcessBuffer	ID3D11Buffer*	Константный буфер для параметров постпроцессинга.	–
m_postProcessEffect	int	Тип эффекта постпроцессинга (0-3).	–
m_detailStrength	float	Сила детализации (влияние detail map).	–
m_pSmallSphere...	Буферы, шейдеры, input layout	Ресурсы для отрисовки маленьких сфер (источников света).	–
m_keyW/A/S/D/Q/E	bool	Флаги состояния клавиш для управления камерой.	–
m_pTerrainVertexBuffer, m_pTerrainIndexBuffer, m_pTerrainGeomBuffer	Буферы вершин, индексов, константный буфер геометрии ландшафта.	–	–
m_terrainGridSizeX/Z, m_terrainWidth/Depth/HeightScale	Параметры сетки ландшафта.	–	–
m_pSceneBuffer	ID3D11Buffer*	Константный буфер сцены (матрица VP, камера, источники света и т.д.).	–

Переменная / Константа	Тип	Назначение	Связь с лекциями
m_pDepthBuffer, m_pDepthStencilView	ID3D11Texture2D*, ID3D11DepthStencilView*	Буфер глубины и его представление.	—
m_pRasterizerState, m_pRasterizerCullFront	ID3D11RasterizerState*	Состояния растеризатора (обычное и для скайбокса).	—
m_pNormalDepthState, m_pSkyboxDepthState	ID3D11DepthStencilState*	Состояния глубины (для reversed depth и для скайбокса).	—
m_pOpaqueBlendState	ID3D11BlendState*	Состояние смешивания (непрозрачные объекты).	—
m_pTexture, m_pTextureView	ID3D11Texture2D*, ID3D11ShaderResourceView*	Основная текстура альbedo.	—
m_pSampler, g_pSkyboxSampler, g_pLinearSampler, g_pLinearMipSampler	ID3D11SamplerState*	Сэмплеры для текстур и IBL.	—
m_width, m_height	UINT	Размеры окна/заднего буфера.	—
m_camPos, m_yaw, m_pitch, m_heightOffset	Параметры камеры.	—	—
m_viewMatrix, m_projMatrix	XMFLOAT4X4	Матрицы вида и проекции.	—
m_rbPressed, m_lbPressed, m_prevMouseX/Y	Состояния мыши.	—	—
g_deltaTime	float	Время между кадрами.	—
g_prevTime, g_freq	LARGE_INTEGER	Для измерения времени.	—

2.2. Структуры данных (объявлены в начале cpp)

Структура	Поля	Назначение	Используется в
SkyboxConstants	world, viewProj, cameraPosAndMode	Константный буфер для скайбокса (матрица мира, VP, позиция камеры).	Render () при отрисовке скайбокса.
GeomBuffer	m, normalM, pbrParams (roughness, metalness, hasNormalMap), posAngle	Константный буфер для каждого объекта (ландшафта). Содержит матрицу модели, матрицу нормалей и PBR-параметры.	TerrainVS.hlsl (b0).
SceneBuffer	vp, cameraPos, lightInfo, lights[10], ambientColor, flowInfo, renderModeInfo, dirLightDir, dirLightColor	Константный буфер сцены (общие данные для всех объектов).	TerrainVS.hlsl и TerrainPS.hlsl (b1).
PostProcessBuffer	effectType, padding[3]	Константный буфер для постпроцессинга (выбранный эффект).	PostProcessPS.hlsl (b0).
SmallSphereGeomBuffer	m (матрица модели), color	Константный буфер для каждой маленькой сферы (источник света).	SmallSphereVS.hlsl (b0).

Структура	Поля	Назначение	Используется в
TextureTangentVertex	<code>x, y, z, tx, ty, tz, nx, ny, nz, u, v</code>	Вершина ландшафта с позицией, касательной, нормалью и текстурными координатами.	Вершинный буфер ландшафта.

Связь с лекциями:

- GeomBuffer и SceneBuffer напрямую соответствуют концепции константных буферов в DirectX, описанной в лекциях по PBR (Лекция 3).
- PostProcessBuffer используется для эффектов постобработки (Лекция 2 о tone mapping, хотя эффекты не описаны в лекциях, но принцип тот же).

2.3. Функции инициализации

2.3.1. WinMain – точка входа

Назначение	Инициализирует COM, создаёт окно, DirectX, загружает ресурсы, запускает главный цикл сообщений.
Как работает	1. CoInitializeEx для WIC. 2. InitWindow – создание окна. 3. InitDirectX – создание устройства, цепочки обмена, буферов. 4. Инициализация таймера. 5. ImGui::CreateContext и инициализация для Win32/DX11. 6. InitShaders, InitTerrain, InitBuffers, загрузка текстур, InitSmallSpheres, InitPostProcess, настройка источников света. 7. Главный цикл while (!exit) – PeekMessage, TranslateMessage, DispatchMessage, вызов Render. 8. Cleanup – освобождение ресурсов.
Связь с лекциями	Инициализация DirectX с флагом D3D11_CREATE_DEVICE_DEBUG в Debug (Лекция 1 – диагностика).

2.3.2. InitWindow – создание окна

Назначение	Регистрирует класс окна, создаёт окно с заданными размерами.
Как работает	Заполняет WNDCLASSEX, регистрирует, создаёт окно с помощью CreateWindow, отображает его.
Связь с лекциями	Стандартная процедура Win32, не связана с графическими лекциями.

2.3.3. InitDirectX – инициализация DirectX 11

Назначение	Создаёт устройство, контекст, цепочку обмена, буфер глубины, состояния, сэмплеры, инициализирует IBL и скайбокс.
Как работает	1. Заполняет DXGI_SWAP_CHAIN_DESC (буферы 2, формат R8G8B8A8_UNORM, FLIP_DISCARD). 2. D3D11CreateDeviceAndSwapChain с аппаратным драйвером или WARP. 3. SetupBackBuffer – создание RTV для заднего буфера и буфера глубины D32_FLOAT. 4. InitColorBuffer – создание промежуточного буфера цвета. 5. Создание состояний растеризатора (CULL_NONE и CULL_FRONT для скайбокса).

Назначение	Создаёт устройство, контекст, цепочку обмена, буфер глубины, состояния, сэмплеры, инициализирует IBL и скайбокс.
	6. Создание состояний глубины для reversed depth (<code>D3D11_COMPARISON_LESS</code> – для reversed depth используется очистка на 1.0 и сравнение LESS, хотя обычно reversed depth использует <code>GREATER_EQUAL</code> , но здесь используется LESS, что не совсем reversed; однако очистка на 1.0 делает дальние объекты ближе к 1, а ближние к 0, и сравнение LESS даёт корректную видимость). 7. Создание состояния смешивания (непрозрачное). 8. Создание сэмплеров: для скайбокса (CLAMP, линейная), для IBL (CLAMP, линейная с поддержкой MIP). 9. Инициализация <code>g_ibl.Init</code> – запуск генерации IBL. 10. Создание буферов скайбокса (сфера) и компиляция шейдеров <code>SkyboxVS/PS</code> .
Связь с лекциями	- Использование <code>D3D11_CREATE_DEVICE_DEBUG</code> – Лекция 1 (диагностика). - Создание сэмплеров с поддержкой MIP для IBL – Лекция 5. - Reversed depth – техника, упомянутая в лекциях по теням (Лекция 8) для повышения точности, но здесь используется для основного рендеринга. - Инициализация IBL – Лекции 4, 5.

2.3.4. `SetupBackBuffer` – создание заднего буфера и буфера глубины

Назначение	Создаёт RTV для заднего буфера и текстуру/DSV для глубины (<code>D32_FLOAT</code>).
Как работает	Получает задний буфер из цепочки обмена <code>GetBuffer</code> , создаёт RTV. Затем создаёт текстуру глубины формата <code>DXGI_FORMAT_D32_FLOAT</code> и DSV.
Связь с лекциями	Базовый этап настройки цели рендера, упоминается в лекциях по началу работы с DirectX.

2.3.5. `InitColorBuffer` – промежуточный буфер цвета

Назначение	Создаёт текстуру для рендеринга сцены до постпроцессинга (цветовой буфер).
Как работает	Создаёт текстуру формата <code>R8G8B8A8_UNORM</code> с флагами <code>D3D11_BIND_RENDER_TARGET D3D11_BIND_SHADER_RESOURCE</code> . Затем создаёт RTV и SRV для этой текстуры.
Связь с лекциями	Используется для реализации постпроцессинга (Лекция 2 – tone mapping, но эффекты не описаны).

2.3.6. `InitShaders` – компиляция шейдеров ландшафта

Назначение	Компилирует <code>TerrainVS.hls1</code> и <code>TerrainPS.hls1</code> , создаёт шейдеры и input layout.
Как работает	Использует <code>D3DCompileFromFile</code> с флагами <code>DEBUG</code> и <code>SKIP_OPTIMIZATION</code> в Debug. Создаёт вершинный и пиксельный шейдеры, а также input layout (позиция, касательная, нормаль, UV).
Связь с лекциями	Компиляция шейдеров с отладочной информацией – Лекция 1 (диагностика). Структура вершин соответствует модели с нормальями и касательными, необходимыми для PBR и карт нормалей (Лекция 3).

2.3.7. `InitTerrain` – генерация ландшафта

Назначение	Загружает карту высот из EXR, создаёт сетку вершин, вычисляет нормали и касательные, создаёт вершинный и индексный буферы, а также константный буфер геометрии.
Как работает	1. <code>LoadEXR</code> из библиотеки <code>TinyEXR</code> загружает float-данные карты высот.

Назначение	Загружает карту высот из EXR, создаёт сетку вершин, вычисляет нормали и касательные, создаёт вершинный и индексный буферы, а также константный буфер геометрии.
	2. Определяет размер сетки с шагом <code>step=2</code> (уменьшение детализации). 3. Заполняет вершины (позиция, UV). 4. Создаёт индексы (два треугольника на ячейку). 5. Вычисляет нормали и касательные: для каждого треугольника вычисляет нормаль как векторное произведение и касательную через UV-производные, накапливает в вершинах, затем нормализует. 6. Сохраняет высоты в <code>m_terrainHeights</code> для интерполяции. 7. Создаёт вершинный и индексный буферы. 8. Создаёт константный буфер <code>GeomBuffer</code> с единичной матрицей.
Связь с лекциями	Генерация нормалей и касательных – основа для PBR и карт нормалей (Лекция 3). Использование EXR для карт высот – не описано в лекциях, но является стандартным подходом.

2.3.8. `LoadTexture` / `LoadNormalMap` / `LoadDetailTexture` / `LoadFlowTexture` / `LoadRoughnessTexture` – загрузка текстур

Функция	Назначение	Как работает	Связь с лекциями
<code>LoadTexture</code>	Загружает альbedo-текстуру из PNG (через <code>LoadPNG</code>), создаёт текстуру с MIP-уровнями в формате <code>R8G8B8A8_UNORM_SRGB</code> (если поддерживается) или <code>UNORM</code> , <code>SRV</code> и сэмплер (анизотропный).	Вызывает <code>LoadPNG</code> , затем создаёт <code>Texture2D</code> с субресурсами для каждого MIP-уровня, создаёт <code>SRV</code> .	Использование формата <code>_SRGB</code> – Лекция 2 (sRGB и гамма-коррекция).
<code>LoadNormalMap</code>	Загружает карту нормалей из DDS (сжатый BC-формат).	Вызывает <code>LoadDDS</code> , создаёт текстуру со сжатым форматом, <code>SRV</code> .	DDS-формат не описан, но является стандартом для текстур в играх.
<code>LoadDetailTexture</code>	Загружает текстуру деталей (PNG) для модуляции альbedo.	Аналогично <code>LoadTexture</code> , но без <code>_SRGB</code> .	–
<code>LoadFlowTexture</code>	Загружает Flow-карту (PNG).	Аналогично.	–
<code>LoadRoughnessTexture</code>	Загружает карту шероховатости (PNG).	Аналогично.	Параметр <code>roughness</code> – ключевой для PBR (Лекция 3).

2.3.9. `InitSmallSpheres` – создание сфер для визуализации источников света

Назначение	Создаёт геометрию маленьких сфер (низкополигональных), шейдеры и константные буферы для отрисовки лампочек в позициях источников света.
Как работает	1. Генерирует сферу через <code>CreateSphere</code> (8x8 сегментов). 2. Создаёт вершинный и индексный буферы. 3. Компилирует простые шейдеры (вершинный – преобразует модель с матрицей, пиксельный – выводит цвет). 4. Создаёт <code>input layout</code> (только позиция). 5. Для каждого источника (до 10) создаёт отдельный константный буфер <code>SmallSphereGeomBuffer</code> .
Связь с лекциями	Не связано с лекциями напрямую, это вспомогательный инструмент для визуализации источников.

2.3.10. InitPostProcess – инициализация постпроцессинга

Назначение	Компилирует шейдеры постпроцессинга (полноэкранный треугольник) и создаёт константный буфер для параметров эффекта.
Как работает	1. Компилирует вершинный шейдер, который генерирует 3 вершины треугольника, покрывающего экран (используя SV_VertexID). 2. Компилирует пиксельный шейдер с тремя эффектами (сепия, холодный тон, ночное зрение). 3. Создаёт динамический константный буфер PostProcessBuffer.
Связь с лекциями	Постпроцессинг – общая техника, описанная в Лекции 2 (tone mapping) и Лекции 7 (Bloom), хотя конкретные эффекты не рассматриваются.

2.4. Функции обновления

2.4.1. UpdateCamera – обновление камеры и константного буфера сцены

Назначение	Обрабатывает ввод с клавиатуры и мыши, вычисляет новые позицию камеры и матрицы вида/проекции, заполняет SceneBuffer и обновляет его.
Как работает	1. Вычисляет deltaTime. 2. По клавишам WASD перемещает камеру в горизонтальной плоскости (с учётом направления взгляда). 3. Q/E – изменяет высоту над рельефом. 4. Ограничивает позицию камеры границами ландшафта. 5. Вычисляет высоту камеры над рельефом через GetTerrainHeight. 6. Формирует матрицу вида XMMatrixLookAtLH. 7. Формирует матрицу проекции с reversed depth (near=0.1, far=100, но очистка глубины на 1.0, сравнение LESS). 8. Заполняет SceneBuffer (матрица VP, позиция камеры, параметры источников, ambient, flow, renderMode, направление и цвет направленного света из IBL). 9. Обновляет m_pSceneBuffer через Map/Unmap.
Связь с лекциями	- Управление камерой – стандартный элемент. - Reversed depth – техника для повышения точности буфера глубины (упоминается в контексте теней, Лекция 8). - Использование направления света из IBL – Лекция 4 (определение направления солнца из HDR). - Передача параметров шероховатости и металличности – Лекция 3.

2.4.2. UpdateLightIntensities – обновление интенсивностей источников в зависимости от Flow-режима

Назначение	Меняет интенсивность точечных источников в зависимости от выбранного режима Flow (ночной режим требует более ярких источников).
Как работает	Если m_flowModeIndex > 0 (ночной режим), то для первого источника интенсивность = 100, для остальных = 20; иначе 80 и 10 соответственно.
Связь с лекциями	Не связано, это логика приложения.

2.4.3. UpdatePostProcessBuffer – обновление константного буфера постпроцессинга

Назначение	Записывает текущий выбранный эффект в PostProcessBuffer и обновляет его через Map/Unmap.
Как работает	Маппит буфер, записывает m_postProcessEffect, анмаппит.

Назначение	Записывает текущий выбранный эффект в <code>PostProcessBuffer</code> и обновляет его через <code>Map/Unmap</code> .
Связь с лекциями	Стандартная работа с динамическими константными буферами.

2.5. Функции рендеринга

2.5.1. `Render` – главная функция рендеринга кадра

Назначение	Выполняет полный цикл рендеринга: обновление, отрисовка сцены в промежуточный буфер, постпроцессинг, <code>ImGui</code> , презентация.
Как работает	1. Вычисляет <code>deltaTime</code>. 2. <code>g_ibl.Update()</code> – если IBL ещё генерируется, делает один шаг. 3. Очищает шейдерные ресурсы. 4. Устанавливает <code>m_pColorBufferRTV</code> как цель рендера и <code>m_pDepthStencilView</code>. 5. Очищает цвет (чёрный) и глубину (1.0 для reversed depth). 6. Устанавливает viewport. 7. <code>UpdateCamera()</code> и <code>UpdatePostProcessBuffer()</code>. 8. Отрисовка скайбокса: если IBL готов, устанавливает специальный depth-стейт (только тест, без записи), растеризатор <code>CULL_FRONT</code>, шейдеры скайбокса, передаёт матрицу view без переноса, текстуру кубмапы (из IBL) и рисует сферу. 9. Отрисовка ландшафта: устанавливает обычный depth-стейт (запись, reversed depth), бленд-стейт (непрозрачный). Передаёт текстуры (альбедо, нормали, детали, flow, roughness), IBL-текстуры (irradiance, prefiltered, BRDF LUT), сэмплеры, константные буферы (геометрический и сцены). Применяет шейдеры ландшафта и рисует индексированную геометрию. 10. Постпроцессинг: <code>RenderPostProcess()</code> – рендерит промежуточный буфер на задний буфер с эффектом. 11. ImGui: если включён, рендерит интерфейс поверх всего. 12. <code>m_pSwapChain->Present(1, 0)</code> – вывод на экран с вертикальной синхронизацией.
Связь с лекциями	- Весь рендеринг соответствует PBR+IBL (Лекции 3,4,5). - Скайбокс – часть IBL (Лекция 4). - Постпроцессинг – Лекция 2 (tone mapping) и Лекция 7 (Bloom – не реализован, но принцип тот же). - Reversed depth – Лекция 8 (тени).

2.5.2. `RenderPostProcess` – применение постэффекта

Назначение	Рендерит полноэкранный треугольник, который сэмплирует промежуточный буфер цвета и применяет выбранный эффект.
Как работает	1. Сбрасывает шейдерный ресурс, чтобы избежать конфликта. 2. Устанавливает задний буфер как цель рендера. 3. Очищает состояния глубины/смешивания/растеризатора (по умолчанию). 4. Устанавливает сэмплер для текстуры. 5. Передаёт промежуточный буфер как SRV (регистр t0). 6. Устанавливает вершинный и пиксельный шейдеры постпроцессинга. 7. Передаёт константный буфер с параметрами. 8. Рисует 3 вершины (треугольник, покрывающий экран).
Связь с лекциями	Реализация постобработки, описанная в Лекции 2 (tone mapping) и Лекции 7 (Bloom).

2.5.3. RenderSmallSpheres – отрисовка лампочек источников

Назначение	Рисует маленькие сферы в позициях источников света (если флаг включён).
Как работает	1. Устанавливает depth-стейт, бленд-стейт. 2. Настраивает шейдеры маленьких сфер, передаёт вершинный/индексный буферы. 3. Для каждого источника обновляет константный буфер (матрица переноса в позицию источника, цвет) и рисует сферу.
Связь с лекциями	Вспомогательная функция, не связана с лекциями.

2.6. Вспомогательные функции

Функция	Назначение	Как работает	Связь с лекциями
CreateSphere	Генерирует вершины и индексы для сферы (заданное число сегментов).	Заполняет массив позиций (нормализованные векторы) и индексы для треугольников.	Используется для создания геометрии маленьких сфер.
CreateSkyboxSphere	Генерирует вершины и индексы для сферы скайбокса (с большим числом сегментов).	Аналогично CreateSphere, но возвращает vector<float> и vector<UINT>.	–
GetTerrainHeight	Интерполирует высоту ландшафта в заданной точке (x,z) по сетке.	Находит четыре ближайших вершины и выполняет билинейную интерполяцию.	–
ResizeSwapChain	Обрабатывает изменение размера окна: пересоздаёт буферы.	Освобождает старые RTV/DSV/цветовой буфер, вызывает ResizeBuffers цепочки обмена, затем SetupBackBuffer и InitColorBuffer.	–
Cleanup	Освобождает все COM-объекты и ресурсы (включая ImGui и IBL).	Вызывает SAFE_RELEASE для всех указателей, завершает ImGui, CoUninitialize.	–
WndProc	Оконная процедура.	Обрабатывает сообщения: WM_SIZE (изменение размера), WM_RBUTTONDOWN/UP, WM_LBUTTONDOWN/UP, WM_MOUSEMOVE, WM_MOUSEWHEEL, WM_KEYDOWN/UP, WM_DESTROY. Также вызывает ImGui_ImplWin32_WndProcHandler.	–

3. Вывод по файлам DirectX11App.cpp и .h

- **DirectX11App.h** – только заглушка.
- **DirectX11App.cpp** – содержит всю логику приложения: инициализацию, главный цикл, рендеринг, обработку ввода. Код хорошо структурирован, использует современные практики (reversed depth, IBL, PBR). Соответствие лекциям прослеживается в реализации PBR, IBL, постпроцессинга, диагностики (debug layer).
- К недостаткам можно отнести большое количество глобальных переменных и отсутствие классов для инкапсуляции (кроме IBL). Однако для учебного проекта это приемлемо.

Детальный разбор файлов `Light.h`, `Texture.h`, `Texture.cpp`

1. Файл `Light.h`

Этот файл содержит определение структуры источника света и вспомогательную функцию расчёта освещения по модели Фонга (используется только для отрисовки маленьких сфер-лампочек, а не для основного PBR-рендеринга).

1.1. Структура `Light`

Элемент	Тип	Назначение	Как работает	Связь с лекциями
<code>pos</code>	<code>DirectX::XMFLOAT4</code>	Позиция источника света в мировых координатах (w=1).	Хранит координаты; используется в шейдерах и при расчёте освещения.	—
<code>color</code>	<code>DirectX::XMFLOAT4</code>	Цвет источника (RGB, альфа игнорируется), значения в диапазоне 0..1.	Умножается на <code>intensity</code> для получения итоговой яркости.	—
<code>intensity</code>	<code>float</code>	Множитель яркости источника.	Позволяет регулировать силу света независимо от цвета.	—
<code>padding[3]</code>	<code>float</code>	Выравнивание до 16 байт (для константного буфера).	Обеспечивает правильное выравнивание структуры в HLSL (16-байтовые границы).	—

Примечание: Структура точно соответствует объявлению в HLSL (в `TerrainPS.hlsl` и `SmallSphereVS.hlsl`), чтобы данные можно было передавать напрямую через константный буфер.

1.2. Функция `CalculateColor`

Параметр	Назначение
<code>objColor</code>	Базовый цвет объекта (альбедо).
<code>objNormal</code>	Нормаль в мировом пространстве.
<code>pos</code>	Позиция пикселя/вершины в мировом пространстве.
<code>cameraPos</code>	Позиция камеры.
<code>lights[10]</code>	Массив источников света.
<code>lightCount</code>	Количество активных источников.
<code>ambientColor</code>	Цвет ambient-освещения.
<code>showNormals</code>	Флаг: если <code>true</code> , возвращает нормаль, преобразованную в цвет (отладка).
<code>shine</code>	Коэффициент блеска для specularной составляющей (если > 0).
Назначение	Вычисляет цвет пикселя по модели Фонга (<code>ambient + diffuse + specular</code>) с квадратичным затуханием. Используется только для отрисовки маленьких сфер, чтобы они были видны как цветные лампочки.

Параметр	Назначение
Как работает	1. Если <code>showNormals == true</code> , возвращает нормаль, преобразованную из диапазона [-1,1] в [0,1]. 2. Начинает с <code>ambientColor * objColor</code> . 3. Для каждого источника: – Вычисляет направление к источнику и расстояние. – Вычисляет квадратичное затухание <code>atten = 1 / dist^2</code>, ограничивая его максимумом 1. – Вычисляет диффузную составляющую как <code>max(dot(lightDir, normal), 0)</code>. – Если <code>shine > 0</code>, вычисляет specularную составляющую: отражает вектор света относительно нормали, берёт скалярное произведение с направлением на камеру, возводит в степень <code>shine</code>. – Прибавляет вклад источника (цвет * интенсивность * диффузная/specularная * затухание). 4. Возвращает итоговый цвет.
Связь с лекциями	Модель Фонга – классическая, но в основном проекте не используется (используется PBR). Данная функция присутствует для визуализации источников света, чтобы они выглядели как светящиеся шарики. Это не связано с лекциями по PBR (Лекция 3), а скорее является вспомогательным инструментом.

2. Файл `Texture.h`

Заголовочный файл для работы с текстурами. Содержит структуру `TextureDesc` и объявления функций загрузки DDS и PNG.

2.1. Структура `TextureDesc`

Поле	Тип	Назначение
<code>pitch</code>	<code>UINT32</code>	Шаг строки в байтах (для несжатых форматов).
<code>mipmapsCount</code>	<code>UINT32</code>	Количество MIP-уровней в текстуре.
<code>fmt</code>	<code>DXGI_FORMAT</code>	Формат пикселей (например, <code>DXGI_FORMAT_R8G8B8A8_UNORM</code>).
<code>width</code>	<code>UINT32</code>	Ширина текстуры.
<code>height</code>	<code>UINT32</code>	Высота текстуры.
<code>pData</code>	<code>void*</code>	Указатель на загруженные данные (выделяется через <code>malloc</code> , должен быть освобождён вызывающим).

Назначение: хранит метаданные и данные текстуры после загрузки из файла. Используется для создания ресурсов DirectX.

2.2. Функции, объявленные в `Texture.h`

Функция	Сигнатура	Назначение
<code>LoadDDS</code>	<code>bool LoadDDS(const std::wstring& filepath, TextureDesc& desc, bool singleMip = false)</code>	Загружает DDS-файл (сжатые BC-форматы) в структуру <code>TextureDesc</code> .
<code>LoadPNG</code>	<code>bool LoadPNG(const std::wstring& filepath, TextureDesc& desc, bool singleMip = false)</code>	Загружает PNG-файл через WIC, генерирует MIP-цепочку вручную (если не <code>singleMip</code>).

Функция	Сигнатура	Назначение
GetBytesPerBlock	UINT32 GetBytesPerBlock(const DXGI_FORMAT& fmt)	Возвращает размер блока (в байтах) для сжатых BC-форматов (8 или 16).
DivUp	UINT32 DivUp(UINT32 a, UINT32 b)	Целочисленное деление с округлением вверх.
WCSToMBS	std::string WCSToMBS(const std::wstring& wstr)	Преобразует широкую строку (wchar_t) в многобайтовую (char).

3. Файл Texture.cpp

Реализация функций для загрузки текстур.

3.1. Вспомогательные функции

Функция	Назначение	Как работает	Связь с лекциями
DivUp	Округление деления вверх.	(a + b - 1) / b.	–
GetBytesPerBlock	Возвращает размер блока для сжатых форматов BC.	switch по DXGI_FORMAT: для BC1 и BC4 возвращает 8, для остальных BC – 16.	–
WCSToMBS	Конвертирует std::wstring в std::string (использует wcstombs_s).	–	–

3.2. Функция LoadDDS

Назначение	Загружает DDS-файл (DirectDraw Surface), поддерживает сжатые форматы BC1, BC2, BC3. Заполняет TextureDesc, выделяет память под данные.
Как работает	1. Открывает файл в бинарном режиме (_w fopen_s). 2. Читает сигнатуру "DDS " (0x20534444). 3. Читает заголовок DDSHeader (размер структуры 124 байта). 4. Проверяет флаги: наличие DDSD_CAPS, DDSD_HEIGHT, DDSD_WIDTH, DDSD_PIXELFORMAT. 5. Определяет формат по четырёмсимвольному коду (fourCC): "DXT1" → DXGI_FORMAT_BC1_UNORM, "DXT3" → BC2_UNORM, "DXT5" → BC3_UNORM. 6. Считывает pitchOrLinearSize (размер данных) или вычисляет его как оставшийся размер файла. 7. Выделяет память malloc(dataSize) и читает все данные (все MIP-уровни) в один буфер. 8. Закрывает файл.
Связь с лекциями	DDS-формат не рассматривается в лекциях, но является стандартным для сжатых текстур в играх. Использование сжатых форматов (BC) упоминается в контексте оптимизации памяти.

3.3. Функция LoadPNG

Назначение	Загружает PNG-файл через Windows Imaging Component (WIC), преобразует в 32-битный RGBA, генерирует MIP-цепочку вручную методом усреднения 2x2. Возвращает все уровни в одном буфере.
Как работает	1. Создаёт фабрику WIC (CoCreateInstance(CLSID_WICImagingFactory)). 2. Создаёт декодер для файла (CreateDecoderFromFilename). 3. Получает первый кадр (GetFrame(0)).

Назначение	Загружает PNG-файл через Windows Imaging Component (WIC), преобразует в 32-битный RGBA, генерирует MIP-цепочку вручную методом усреднения 2x2. Возвращает все уровни в одном буфере.
	4. Получает размеры изображения. 5. Создаёт формат-конвертер для преобразования в GUID_WICPixelFormat32bppRGBA. 6. Копирует данные первого уровня (MIP 0) в <code>std::vector<BYTE></code> . 7. Вычисляет количество MIP-уровней: пока ширина или высота > 1, делим пополам. 8. Генерирует последующие MIP-уровни вручную: для каждого пикселя нового уровня усредняет 2x2 блока из предыдущего уровня (по 4 каналам). 9. Собирает все уровни в один буфер (в порядке от большего к меньшему). 10. Выделяет память <code>malloc</code> и копирует данные. 11. Освобождает COM-объекты.
Связь с лекциями	- Использование sRGB-формата для альбедо (в <code>DirectX11App.cpp</code> при создании текстуры) – соответствует Лекции 2 (гамма-коррекция). - Генерация MIP-уровней вручную – стандартная техника для уменьшения алиасинга при удалении объектов (упоминается в контексте текстур, но не в конкретной лекции).

4. Общий комментарий по файлам

- **Light.h** – минимален, содержит только структуру источника и функцию Фонга (используется исключительно для визуализации). В основном проекте PBR-освещение реализовано в шейдерах.
- **Texture.h/cpp** – обеспечивают загрузку текстур из DDS и PNG. DDS-загрузка поддерживает только BC1/BC2/BC3 (наиболее распространённые). PNG-загрузка использует WIC и генерирует MIP-уровни программно (что может быть медленным для больших текстур, но для учебного проекта приемлемо). Обе функции возвращают данные в формате, готовом для создания DirectX-текстур (с учётом выравнивания для сжатых форматов).

Связь с лекциями:

- Прямых ссылок на конкретные лекции нет, но использование сжатых форматов и управление MIP-уровнями является стандартной практикой, упоминаемой в различных разделах (оптимизация, качество текстур).
- Форматы `_SRGB` для альбедо и линейные для нормалей/roughness соответствуют рекомендациям Лекции 2 (работа с цветовыми пространствами).
- В проекте используются текстуры высокого разрешения (4K), что требует эффективной загрузки и управления памятью.

Детальный разбор файлов IBL.h, IBL.cpp И PBR.hlsl

1. Файл IBL.h

Заголовочный файл класса IBL, отвечающего за генерацию всех ресурсов для Image-Based Lighting: кубическая карта из HDR, irradiance map, prefiltered environment map, BRDF LUT.

1.1. Структура IBLResources

Поле	Тип	Назначение
hdrSRV	ID3D11ShaderResourceView*	Шейдерное представление загруженной HDR-текстуры (2D, формат R32G32B32A32_FLOAT). Используется для генерации кубмапы.
skyboxSRV	ID3D11ShaderResourceView*	Шейдерное представление кубической карты для скайбокса (это та же карта, что и cubeMapSRV, но отдельный указатель).
cubeMap	ID3D11Texture2D*	Текстура кубической карты (6 граней, с MIP-уровнями).
cubeMapSRV	ID3D11ShaderResourceView*	SRV для cubeMap.
irradianceMap	ID3D11Texture2D*	Текстура irradiance map (кубическая, низкого разрешения).
irradianceSRV	ID3D11ShaderResourceView*	SRV для irradiance map.
prefilteredMap	ID3D11Texture2D*	Текстура prefiltered environment map (кубическая, с MIP-уровнями для разных roughness).
prefilteredSRV	ID3D11ShaderResourceView*	SRV для prefiltered map.
brdfLUT	ID3D11Texture2D*	2D-текстура BRDF LUT (формат R16G16_FLOAT).
brdfLUTSRV	ID3D11ShaderResourceView*	SRV для BRDF LUT.

Назначение: хранит все текстурные ресурсы, сгенерированные IBL, для использования в основном рендере.

1.2. Класс IBL

Элемент	Тип	Назначение
m_device, m_context	ID3D11Device*, ID3D11DeviceContext*	Указатели на устройство и контекст (передаются в Init).
m_resources	IBLResources	Хранит созданные текстуры.
m_stage	IBLStage (enum)	Текущая стадия генерации: NONE, CUBEMAP, IRRADIANCE, PREFILTER, BRDF, DONE.

Элемент	Тип	Назначение
m_currentFace	int	Номер текущей грани кубмапы (0-5) при генерации.
m_currentMip	int	Номер текущего MIP-уровня при генерации prefiltered map.
m_projCubemap	XMMATRIX	Матрица проекции для рендеринга граней кубмапы (90° FOV, 0.1..10).
m_sunDirection	XMFLOAT3	Направление на самый яркий источник в HDR (луна/солнце), вычисляемое при инициализации.
m_irradianceCB,m_prefilterCB,m_pCubeMapCB	ID3D11Buffer*	Константные буферы для шейдеров генерации.
Константы CUBEMAP_RES, IRRADIANCE_RES, PREFILTERED_RES, PREFILTERED_MIPS, BRDF_LUT_RES	static const UINT	Разрешения текстур и количество MIP-уровней для prefiltered.

Методы:

Метод	Сигнатура	Назначение
Init	bool Init(ID3D11Device* device, ID3D11DeviceContext* context)	Инициализирует IBL: загружает HDR, создаёт текстуры, запускает пошаговую генерацию.
Update	void Update()	Выполняет один шаг генерации (одна грань или один MIP-уровень) за вызов.
Shutdown	void Shutdown()	Освобождает все ресурсы.
GetResources	const IBLResources& GetResources() const	Возвращает структуру с ресурсами.
IsReady	bool IsReady() const	Возвращает true, если генерация завершена.
GetSunDirection	XMFLOAT3 GetSunDirection() const	Возвращает направление на солнце/луну.

Приватные методы:

Метод	Назначение
RenderCubemapFace()	Рендерит одну грань кубмапы из HDR-панорамы.
RenderIrradianceFace()	Рендерит одну грань irradiance map (интегрирование по полусфере).
RenderPrefilterStep()	Рендерит один MIP-уровень одной грани prefiltered map (importance sampling).
RenderBRDF()	Генерирует 2D-текстуру BRDF LUT.
LoadHDRTexture(const wchar_t* filename)	Загружает HDR-файл через stb_image.

Метод	Назначение
NextStage()	Переключает стадию (не используется напрямую, логика в Update).

Связь с лекциями:

- Весь класс реализует алгоритмы IBL из Лекции 4 (диффузный IBL) и Лекции 5 (спекулярный IBL).
- Конкретные шаги: преобразование equirectangular → cubemap, вычисление irradiance map, prefiltered env map, BRDF LUT.

2. Файл `IBL.cpp`

Реализация методов класса `IBL`. Здесь также содержатся глобальные статические переменные (шейдеры, сэмплеры, матрицы видов для граней куба) и вспомогательные функции.

2.1. Глобальные статические данные

Переменная	Тип	Назначение
<code>g_cubeViews[6]</code>	<code>XMMATRIX</code>	Матрицы вида для каждой из 6 граней кубмапы (направления: +X, -X, +Y, -Y, +Z, -Z).
<code>g_pFullscreenQuadVS, g_pHDRToCubeVS, g_pHDRToCubePS, g_pIrradiancePS, g_pPrefilterPS, g_pBRDFPS</code>	Шейдеры	Компилированные шейдеры для генерации.
<code>g_pLinearSampler, g_pLinearMipSampler, g_pIrradianceSampler</code>	<code>ID3D11SamplerState*</code>	Сэмплеры для разных этапов.

2.2. Вспомогательные функции (в `IBL.cpp`)

Функция	Назначение	Как работает	Связь с лекциями
<code>FindSunDirectionFromHDR</code>	Находит самый яркий пиксель в HDR-панораме и определяет направление (для направленного света).	1. Проходит по пикселям с шагом 8, вычисляет яркость (luminance). 2. Находит максимальную яркость, запоминает координаты. 3. Переводит координаты пикселя в сферические углы (u,v) → theta, phi. 4. Вычисляет декартово направление и нормализует. 5. Инвертирует направление (потому что из HDR мы получаем направление на источник, а нужен вектор от источника).	Упоминается в Лекции 4 как способ получить направление солнца из HDR.
<code>LoadCSO</code>	Загружает скомпилированный шейдер из CSO-файла.	<code>D3DReadFileToBlob</code> .	—
<code>CreateCubeMapTexture</code>	Создаёт кубическую текстуру с заданными размерами, количеством MIP-уровней и форматом, а также SRV.	Заполняет <code>D3D11_TEXTURE2D_DESC</code> с <code>ArraySize=6</code> , флагом <code>D3D11_RESOURCE_MISC_TEXTURECUBE</code> и <code>D3D11_RESOURCE_MISC_GENERATE_MIPS</code> (для автогенерации MIP после заполнения).	—
<code>CreateFaceRTV</code>	Создаёт RTV для конкретной грани и MIP-уровня кубмапы.	Использует <code>D3D11_RTV_DIMENSION_TEXTURE2DARRAY</code> , задаёт <code>FirstArraySlice=face</code> , <code>MipSlice=mip</code> .	—
<code>DrawFullscreenQuad</code>	Рисует полноэкранный квадрат (4 вершины,	<code>IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP), Draw(4, 0)</code> .	—

Функция	Назначение	Как работает	Связь с лекциями
	TRIANGLESTRIP) без индексного буфера.		

2.3. Методы класса IBL

2.3.1. Init

Назначение	Инициализирует IBL: загружает HDR, создаёт текстуры, запускает генерацию.
Как работает	1. Загружает шейдеры CSO через LoadCSO и создаёт их. 2. Создаёт сэмплеры: - g_pLinearSampler – линейная фильтрация, clamp, MaxLOD = 0 (только уровень 0). - g_pLinearMipSampler – линейная фильтрация с поддержкой MIP (MaxLOD = FLT_MAX). - g_pIrradianceSampler – линейная фильтрация, MaxLOD = 0 (без MIP). 3. Загружает HDR-файл через stb_image (stbi_loadf), получает float* данные. 4. Вычисляет m_sunDirection через FindSunDirectionFromHDR. 5. Создаёт 2D-текстуру HDR (формат R32G32B32A32_FLOAT, Usage=IMMUTABLE) и SRV. 6. Вычисляет количество MIP-уровней для кубмапы (исходя из CUBEMAP_RES). 7. Создаёт кубмапу (CreateCubeMapTexture) с этим количеством MIP-уровней и форматом R16G16B16A16_FLOAT. 8. Устанавливает m_resources.skyboxSRV = m_resources.cubeMapSRV (с AddRef). 9. Создаёт irradiance map (IRRADIANCE_RES x IRRADIANCE_RES, 1 MIP-уровень), prefilered map (PREFILTERED_RES, PREFILTERED_MIPS MIP-уровней). 10. Создаёт константные буферы для шейдеров. 11. Устанавливает m_stage = IBL_STAGE_CUBEMAP, m_currentFace = 0, m_currentMip = 0.
Связь с лекциями	Весь процесс соответствует алгоритмам из Лекций 4 и 5. Использование R16G16B16A16_FLOAT для кубмап – экономия памяти по сравнению с полным float.

2.3.2. Update

Назначение	Выполняет один шаг генерации в зависимости от текущей стадии.
Как работает	switch (m_stage): - CUBEMAP: вызывает RenderCubemapFace(). - IRRADIANCE: вызывает RenderIrradianceFace(). - PREFILTER: вызывает RenderPrefilterStep(). - BRDF: вызывает RenderBRDF(), затем устанавливает m_stage = DONE. По окончании каждой стадии увеличиваются счётчики граней/MIP-уровней, и при завершении всех граней/уровней переключается на следующую стадию.
Связь с лекциями	Пошаговое выполнение предотвращает TDR (Timeout Detection Recovery) – упоминается в Лекции 1 (диагностика) как способ борьбы с зависаниями при долгих вычислениях на GPU.

2.3.3. RenderCubemapFace

Назначение	Рендерит одну грань кубмапы из HDR-панорамы.
------------	--

Назначение	Рендерит одну грань кубмапы из HDR-панорамы.
Как работает	1. Создаёт RTV для текущей грани (m_currentFace) и MIP 0 через CreateFaceRTV. 2. Очищает RTV (чёрный). 3. Отвязывает все входные ресурсы (чтобы избежать конфликта). 4. Устанавливает цель рендера – RTV. 5. Устанавливает вершинный шейдер g_pHDRToCubeVS и пиксельный g_pHDRToCubePS. 6. Устанавливает сэмплер g_pLinearSampler. 7. Передаёт HDR-текстуру как SRV (регистр t0). 8. Вычисляет обратную матрицу invViewProj для текущей грани (view * proj, затем инвертирование и транспонирование) и обновляет константный буфер m_pCubeMapCB. 9. Устанавливает viewport под размер CUBEMAP_RES. 10. Вызывает DrawFullscreenQuad. 11. Освобождает RTV. 12. После обработки всех 6 граней вызывает m_context->GenerateMips(m_resources.cubeMapSRV) для автогенерации MIP-уровней, затем переключает стадию на IRRADIANCE.
Связь с лекциями	Преобразование equirectangular → cubemap описано в Лекции 4. Используется шейдер HDRToCubeMap_PS.hlsl, который по направлению вычисляет UV-координаты для сэмплинга HDR-панорамы.

2.3.4. RenderIrradianceFace

Назначение	Рендерит одну грань irradiance map (диффузный IBL).
Как работает	1. Создаёт RTV для текущей грани и MIP 0 на irradianceMap. 2. Очищает, отвязывает ресурсы. 3. Устанавливает шейдеры: VS – g_pHDRToCubeVS, PS – g_pIrradiancePS. 4. Устанавливает сэмплер g_pIrradianceSampler (без MIP). 5. Передаёт кубмапу (cubeMapSRV) как входную текстуру. 6. Обновляет константный буфер m_pCubeMapCB (обратная матрица view-proj). 7. Обновляет константный буфер m_irradianceCB с параметрами Nphi и Ntheta (количество семплов по азимуту и полярному углу). В коде задано 256 и 128 соответственно. 8. Устанавливает viewport под размер IRRADIANCE_RES. 9. Рисует полноэкранный квад. 10. После всех 6 граней переключает стадию на PREFILTER.
Связь с лекциями	Интегрирование по полусфере с использованием метода Римана – описано в Лекции 4. Количество семплов влияет на качество. В коде используется 256x128 семплов, что обеспечивает хорошее качество.

2.3.5. RenderPrefilterStep

Назначение	Рендерит один MIP-уровень одной грани prefiltered environment map (спекулярный IBL).
Как работает	1. Создаёт RTV для текущей грани и текущего MIP-уровня (m_currentMip) на prefilteredMap. 2. Очищает, отвязывает ресурсы. 3. Устанавливает шейдеры: VS – g_pHDRToCubeVS, PS – g_pPrefilterPS. 4. Устанавливает сэмплер g_pLinearMipSampler (поддержка MIP). 5. Передаёт кубмапу (cubeMapSRV) как входную текстуру. 6. Обновляет m_pCubeMapCB (обратная матрица view-proj).

Назначение	Рендерит один MIP-уровень одной грани <code>prefiltered environment map</code> (спекулярный IBL).
	7. Обновляет <code>m_prefilterCB</code> со значением <code>roughness = m_currentMip / (PREFILTERED_MIPS - 1)</code> . 8. Устанавливает <code>viewport</code> под размер текущего MIP-уровня (<code>PREFILTERED_RES >> m_currentMip</code>). 9. Рисует полноэкранный квад. 10. После обработки всех 6 граней текущего MIP-уровня переходит к следующему MIP-уровню. После всех MIP-уровней переключает стадию на <code>BRDF</code> .
Связь с лекциями	Importance sampling с распределением GGX – подробно описано в Лекции 5. В шейдере <code>PrefilterEnvMap_PS.hlsl</code> реализована генерация выборок по Хаммерсли, вычисление PDF и выборка с уровнем MIP на основе разрешения.

2.3.6. RenderBRDF

Назначение	Генерирует 2D-текстуру BRDF LUT (scale и bias) размером <code>BRDF_LUT_RES</code> x <code>BRDF_LUT_RES</code> .
Как работает	1. Создаёт 2D-текстуру формата <code>R16G16_FLOAT</code> с флагами <code>RENDER_TARGET</code> и <code>SHADER_RESOURCE</code> . 2. Создаёт RTV и SRV для неё. 3. Очищает RTV (чёрный). 4. Устанавливает шейдеры: <code>VS – g_pFullscreenQuadVS</code> , <code>PS – g_pBRDFPS</code> . 5. Рисует полноэкранный квад (без входных текстур). 6. Освобождает RTV. 7. Переключает стадию на <code>DONE</code> .
Связь с лекциями	BRDF LUT вычисляется в шейдере <code>BRDFLUT_PS.hlsl</code> с использованием importance sampling и интегрирования по BRDF для всех комбинаций <code>NdotV</code> и <code>roughness</code> . Описано в Лекции 5.

2.3.7. Shutdown

Назначение	Освобождает все ресурсы, созданные в IBL (текстуры, SRV, буферы, сэмплеры).
Как работает	Вызывает <code>SAFE_RELEASE</code> для всех указателей в <code>m_resources</code> , а также для константных буферов и глобальных шейдеров/сэмплеров.

3. Файл `PBR.hlsl`

Общий заголовочный файл с функциями для PBR-модели Кука-Торранса. Включается в основной пиксельный шейдер `TerrainPS.hlsl`.

3.1. Константы

Константа	Значение	Назначение
PI	3.14159265358979323846	Число π, используется в формулах.

3.2. Функции

Функция	Сигнатура	Назначение	Формула	Связь с лекциями
---------	-----------	------------	---------	------------------

Функция	Сигнатура	Назначение	Формула	Связь с лекциями
D_GGX	float D_GGX(float NdotH, float roughness)	Normal Distribution Function (GGX / Trowbridge-Reitz).	$D = \frac{\alpha^2 \pi \cdot ((N \cdot H)^2 \cdot (\alpha^2 - 1) + 1)}{2}$ $D = \pi \cdot ((N \cdot H)^2 \cdot (\alpha^2 - 1) + 1) 2 \alpha^2$, где $\alpha = roughness$ $\alpha = roughness$.	Лекция 3, слайд 34.
G_SchlickGGX	float G_SchlickGGX(float NdotV, float roughness)	Geometry Function (Schlick-GGX) для одного направления.	$G_{Schlick} = N \cdot V (N \cdot V) \cdot (1 - k) + k$ $G_{Schlick} = (N \cdot V) \cdot (1 - k) + k N \cdot V$, где $k = \frac{\alpha + 1}{2}$ $k = 8 \frac{\alpha + 1}{2}$.	Лекция 3, слайд 37.
G_Smith	float G_Smith(float NdotV, float NdotL, float roughness)	Smith Geometry Function (произведение для двух направлений).	$G = G_{Schlick}(N, V) \cdot G_{Schlick}(N, L)$ $G = G_{Schlick}(N, V) \cdot G_{Schlick}(N, L)$.	Лекция 3, слайд 37.
F_Schlick	float3 F_Schlick(float3 F0, float VdotH)	Fresnel-Schlick для точечных источников.	$F = F_0 + (1 - F_0) \cdot (1 - V \cdot H)^5$ $F = F_0 + (1 - F_0) \cdot (1 - V \cdot H)^5$.	Лекция 3, слайд 40.
FresnelSchlickRoughness	float3 FresnelSchlickRoughness(float3 F0, float roughness, float3 V, float3 N)	Fresnel-Schlick для IBL с учётом шероховатости.	$F = F_0 + (\max(1 - roughness, F_0) - F_0) \cdot (1 - N \cdot V)^5$ $F = F_0 + (\max(1 - roughness, F_0) - F_0) \cdot (1 - N \cdot V)^5$.	Лекция 4, слайд 31 (аппроксимация для IBL).
GetF0	float3 GetF0(float3 albedo, float metalness, float dielectricF0)	Вычисляет базовое отражение F0 из параметров материала.	<code>lerp(dielectricF0.xxx, albedo, metalness).</code>	Лекция 3, слайд 43 (использование metalness для смешивания).

Общее назначение: предоставляет математические функции, используемые в PBR-шейдерах для расчёта диффузной и specularной составляющих.

4. Сводка по IBL-генерации

Стадия	Шейдер	Вход	Выход	Ключевые параметры	Связь с лекцией
Cubemap	HDRToCubeMap_PS	HDR-панорама (2D)	Кубическая карта (6 граней, 256x256, FP16)	Обратная матрица view-proj; преобразование направление→UV.	Лекция 4 (преобразование).
Irradiance	IrradianceMap_PS	Кубическая карта	Irradiance map (128x128, FP16)	Интегрирование по полусфере с 256x128 семплами; нормировка на π.	Лекция 4 (диффузный IBL).
Prefiltered	PrefilterEnvMap_PS	Кубическая карта	Prefiltered map (256x256, 6 MIP-уровней, FP16)	Importance sampling GGX с 256 семплами; вычисление mip-уровня на основе PDF и разрешения.	Лекция 5 (specularный IBL).
BRDF LUT	BRDFLUT_PS	Нет (только UV координаты)	2D LUT (128x128, R16G16_FLOAT)	Интегрирование BRDF с 1024 семплами для каждой комбинации NdotV и roughness.	Лекция 5 (BRDF LUT).

5. Заключение

- IBL.h/cpp** – полноценная реализация генерации IBL-ресурсов, соответствующая теоретическим основам из лекций 4 и 5. Пошаговое выполнение предотвращает TDR.
- PBR.hlsl** – коллекция функций для PBR-модели, используемая во всех шейдерах. Соответствует лекции 3.

- В коде используются современные практики: importance sampling, генерация MIP-цепочки, сжатые форматы (FP16 для кубмап).

Детальный разбор шейдеров: TerrainVS.hlsl, TerrainPS.hlsl, SkyboxVS.hlsl, SkyboxPS.hlsl

1. TerrainVS.hlsl – вершинный шейдер ландшафта

Назначение: преобразует вершины ландшафта из локального пространства в мировое и экранное, вычисляет касательный базис (нормаль, касательная) и передаёт в пиксельный шейдер параметры материала (roughness, metalness, наличие карты нормалей). Также передаёт UV-координаты.

1.1. Константные буферы

Буфер	Регистр	Поля	Назначение
GeomBuffer	b0	float4x4 m – матрица модели (локальное → мировое) float4x4 normalM – матрица для преобразования нормалей и касательных float4 pbrParams – (x: roughness, y: metalness, z: unused, w: hasNormalMap) float4 posAngle – не используется	Индивидуальные параметры каждого объекта (ландшафта).
SceneBuffer	b1	float4x4 vp – матрица вида-проекции float4 cameraPos – позиция камеры ... (остальные поля не используются в VS)	Общие параметры сцены.

1.2. Входные и выходные данные

Структура	Элементы	Назначение
VSInput	float3 pos : POSITION float3 tangent : TANGENT float3 norm : NORMAL float2 uv : TEXCOORD	Вершинный буфер (генерируется в InitTerrain).
VSOutput	float4 pos : SV_POSITION – позиция в clip-space float3 worldPos : POSITION – позиция в мировом пространстве float3 tangent : TANGENT – касательная в мировом пространстве float3 norm : NORMAL – нормаль в мировом пространстве float2 uv : TEXCOORD – текстурные координаты nointerpolation float4 geomData : GEOM_DATA – (x: roughness, y: metalness, w: hasNormalMap)	Передаётся в пиксельный шейдер.

1.3. Основная функция main

Шаг	Действие	Пояснение
1	float4 worldPos = mul(float4(vertex.pos, 1.0), m);	Преобразует позицию из локального пространства в мировое.
2	result.pos = mul(worldPos, vp);	Преобразует мировую позицию в clip-space (умножение на матрицу вида-проекции).
3	result.worldPos = worldPos.xyz;	Сохраняет мировую позицию для дальнейшего использования (освещение, IBL).

Шаг	Действие	Пояснение
4	<code>result.tangent = mul(float4(vertex.tangent, 0.0f), normalM).xyz;</code>	Преобразует касательную в мировое пространство с использованием матрицы нормалей (без переноса).
5	<code>result.norm = mul(float4(vertex.norm, 0.0f), normalM).xyz;</code>	Преобразует нормаль в мировое пространство.
6	<code>result.uv = vertex.uv;</code>	Передаёт текстурные координаты.
7	<code>result.geomData = pbrParams;</code>	Передаёт параметры материала (roughness, metalness, hasNormalMap).

Особенности:

- Матрица `normalM` должна быть обратной транспонированной матрицей модели для корректного преобразования нормалей. В данном случае ландшафт имеет единичную матрицу модели, поэтому `normalM` – единичная матрица.
- Используется `nointerpolation` для `geomData`, так как это константные значения для всего объекта (не интерполируются между вершинами).

Связь с лекциями:

- Вершинный шейдер выполняет стандартные преобразования, описанные в основах графики.
- Передача касательной и нормали необходима для PBR и карт нормалей (Лекция 3, слайды о TBN-матрице).
- `nointerpolation` – техника для передачи константных данных на весь примитив.

2. `TerrainPS.hlsl` – пиксельный шейдер ландшафта

Назначение: выполняет полный PBR+IBL-рендеринг ландшафта с учётом текстур (альбедо, нормали, детали, flow, roughness), Flow-эффекта, направленного и точечного освещения, отладочных режимов, тонального отображения и гамма-коррекции.

2.1. Текстуры и сэмплеры

Регистр	Текстура	Назначение
t0	<code>colorTexture</code>	Альбедо (основная текстура, sRGB).
t1	<code>normalMapTexture</code>	Карта нормалей (линейное пространство).
t2	<code>detailTexture</code>	Карта деталей (модулирует альбедо).
t3	<code>flowTexture</code>	Карта Flow (маска для эффекта потоков).
t4	<code>roughnessTexture</code>	Карта шероховатости (линейное пространство).
t5	<code>metalnessTexture</code>	Карта металличности (не используется, зарезервирована).
t6	<code>DiffuseIrradianceMap</code>	Irradiance map (диффузный IBL).
t7	<code>PrefilteredEnvMap</code>	Prefiltered environment map (спекулярный IBL).

Регистр	Текстура	Назначение	
t8	BRDFLut	BRDF LUT (scale и bias).	
Регистр	Сэмплер	Назначение	
s0	colorSampler	Основной сэмплер (анизотропный) для текстур.	
s1	linearSampler	Линейный сэмплер без MIP (для irradiance и BRDF LUT).	
s2	linearMipSampler	Линейный сэмплер с поддержкой MIP (для prefiltered).	

2.2. Константные буферы (используемые в PS)

Буфер	Регистр	Поля, используемые в PS
SceneBuffer	b1	cameraPos – позиция камеры lightInfo – (x: lightCount, y: useNormalMaps, z: showNormals, w: detailStrength) lights[10] – массив точечных источников ambientColor – цвет ambient (не используется, т.к. есть IBL) flowInfo – (x: flowMode) renderModeInfo – (x: renderMode) dirLightDir – направление направленного света dirLightColor – цвет и интенсивность направленного света

(Также есть manualPBRParams – закомментирован, не используется.)

2.3. Входные данные пиксельного шейдера (vsOutput)

Поле	Назначение
pixel.pos	Clip-space позиция (не используется).
pixel.worldPos	Мировая позиция пикселя.
pixel.tangent	Касательная в мировом пространстве.
pixel.norm	Нормаль в мировом пространстве (интерполированная).
pixel.uv	Текстурные координаты.
pixel.geomData	(roughness, metalness, hasNormalMap) – константы для всего объекта.

2.4. Основные этапы работы main

2.4.1. Сэмплирование текстур

Действие	Пояснение
<code>float3 albedo = colorTexture.Sample(colorSampler, pixel.uv).rgb;</code>	Чтение альбеда. Текстура загружена в формате <code>_SRGB</code> , поэтому возвращает линейные значения.
<code>float3 detail = detailTexture.Sample(...).rgb;</code>	Чтение карты деталей.
<code>float3 flow = flowTexture.Sample(...).rgb;</code>	Чтение Flow-карты.
<code>float roughness = ...;</code>	Чтение <code>roughness</code> из текстуры (красный канал), затем инвертирование ($1.0 - r$), масштабирование и <code>clamp</code> . Комментарий: в коде <code>roughness = 1.0 - roughnessTexture.Sample(...).r</code> ; – это означает, что в текстуре хранится "гладкость" (<code>glossiness</code>), а не шероховатость, поэтому инвертируется. Затем умножается на 0.5 и <code>clamp</code> [0.05, 1.0].
<code>metalness = 0.0;</code>	Фиксированное значение (ландшафт – диэлектрик).

2.4.2. Построение нормали (с учётом карты нормалей)

Действие	Пояснение
<code>float3 N = normalize(pixel.norm);</code>	Интерполированная нормаль.
<code>if (lightInfo.y > 0.5 && pixel.geomData.w > 0.5)</code>	Если карта нормалей включена и доступна: <code>float3 texNormal = normalMapTexture.Sample(...).rgb * 2.0 - 1.0;</code> – распаковка из [0,1] в [-1,1]. <code>float3 bitangent = normalize(cross(pixel.tangent, pixel.norm));</code> – вычисление бинормали. <code>float3x3 TBN = float3x3(pixel.tangent, bitangent, pixel.norm);</code> – матрица TBN. <code>N = normalize(mul(texNormal, TBN));</code> – преобразование нормали из <code>tangent space</code> в <code>world space</code> .
Связь с лекциями	Использование TBN-матрицы для карт нормалей – Лекция 3 (PBR).

2.4.3. Flow-эффект (модификация материала)

Режим (flowMode)	Изменения	Пояснение
1 – Влажные ущелья	<code>albedo *= 0.35</code> (затемнение) <code>roughness = lerp(roughness, 0.15, wetMask)</code> <code>dielectricF0 = lerp(0.04, 0.08, wetMask)</code> <code>specularIblFactor = lerp(1.0, 0.30, wetMask)</code> <code>occlusionFactor = 1.0 - flowMask</code> (с <code>lerp</code>)	Имитация мокрого камня: темнее, более гладкий, повышенный F0, ослабленный specular IBL.
2 – Каменистое дно	<code>albedo *= 0.45</code> <code>roughness = lerp(roughness, 0.85, flowMask)</code> <code>occlusionFactor = lerp(1.0, 0.15, flowMask)</code> <code>specularIblFactor = lerp(1.0, 0.05, flowMask)</code>	Глубокие сухие расщелины: очень тёмные, матовые, сильная окклюзия.
3 – Горные ручьи	<code>waterMask = pow(flowMask, 4.0)</code> <code>albedo *= 0.10</code>	Тонкие зеркальные жилы воды: очень гладкие, низкий F0 (вода), прозрачные.

Режим (flowMode)	Изменения	Пояснение
	roughness = lerp(roughness, 0.02, waterMask) dielectricF0 = lerp(0.04, 0.02, waterMask) occlusionFactor = lerp(1.0, 0.20, waterMask) specularIblFactor = lerp(1.0, 0.10, waterMask)	

Flow-маска вычисляется как яркость пикселя в flowTexture, затем возводится в степень для сужения (зависит от режима). occlusionFactor и specularIblFactor применяются позже для IBL-компонент.

2.4.4. Вычисление F0 и подготовка

Действие	Пояснение
float3 F0 = GetF0(albedo, metalness, dielectricF0);	Вызов функции из PBR.hlsl. Для диэлектриков (metalness=0) F0 = dielectricF0 (0.04 или изменённый Flow).
float NdotV = max(dot(N, V), 0.0);	Косинус угла между нормалью и направлением взгляда.
float3 R = normalize(reflect(-V, N));	Направление отражения (для specular IBL).

2.4.5. IBL – диффузная и specular составляющие

Компонента	Вычисление	Пояснение
Диффузный IBL	float3 irradiance = DiffuseIrradianceMap.SampleLevel(linearSampler, N, 0).rgb; float3 kD = (1.0 - F) * (1.0 - metalness); float3 diffuseIBL = irradiance * albedo * kD;	Irradiance map сэмплируется по направлению нормали. F – Fresnel для IBL (с учётом roughness). kD – доля диффузного отражения (сохранение энергии).
Спекулярный IBL	float3 prefilteredColor = PrefilteredEnvMap.SampleLevel(linearMipSampler, R, roughness * (PREFILTERED_MIPS - 1)).rgb; float2 brdf = BRDFLut.Sample(linearSampler, float2(NdotV, roughness)).rg; float3 specularIBL = prefilteredColor * (F0 * brdf.x + brdf.y);	Prefiltered env map сэмплируется по направлению отражения с уровнем MIP, зависящим от roughness. BRDF LUT даёт scale и bias. Результат – specular IBL.
Применение Flow-коэффициентов	diffuseIBL *= occlusionFactor; specularIBL *= specularIblFactor;	Flow-эффект модулирует интенсивность IBL для затемнения в расщелинах.
Ночное затухание	diffuseIBL *= 0.03f; specularIBL *= 0.12f;	Искусственное ослабление для ночной сцены (итерация 4).
ambientIBL = diffuseIBL + specularIBL;	Сумма → добавляется к итоговому цвету.	

Связь с лекциями: полная реализация IBL из Лекций 4 (диффузный) и 5 (specularный).

2.4.6. Направленный свет (солнце/луна)

Действие	Пояснение
float3 L_dir = normalize(-dirLightDir.xyz);	Направление от поверхности к источнику.

Действие	Пояснение
<code>float NdotL_dir = max(dot(N, L_dir), 0.0);</code>	Диффузный фактор.
<code>float3 radiance_dir = dirLightColor.rgb * dirLightColor.w;</code>	Цвет * интенсивность.
Вычисление <code>H_dir, NdotH_dir, VdotH_dir, NdotV_dir</code> .	Промежуточные значения для PBR.
<code>D_dir = D_GGX(NdotH_dir, roughness);</code>	NDF.
<code>G_dir = G_Smith(NdotV_dir, NdotL_dir, roughness);</code>	Geometry.
<code>F_dir = F_Schlick(F0, VdotH_dir);</code>	Fresnel.
<code>float3 specular_dir = (D_dir * G_dir * F_dir) / max(4.0 * NdotV_dir * NdotL_dir, 0.001);</code>	Спекулярная часть Cook-Torrance.
<code>float3 kD_dir = (1.0 - F_dir) * (1.0 - metalness);</code>	Диффузная часть (сохранение энергии).
<code>color += (kD_dir * albedo / PI + specular_dir) * radiance_dir * NdotL_dir;</code>	Добавление вклада направленного света.

Направленный свет используется как дополнительный источник (луна) с параметрами, заданными в SceneBuffer.

2.4.7. Точечные источники (в коде закомментированы, но структура есть)

В коде точечные источники не добавляются, т.к. они были заменены IBL и направленным светом. Однако массив `lights` присутствует в SceneBuffer и мог бы быть использован по аналогии с направленным светом.

2.4.8. Отладочные режимы (`renderMode`)

Режим	Отображаемая компонента	Пояснение
0	Полный PBR+IBL	Обычный рендеринг.
1	D (NDF)	Normal Distribution Function.
2	G (Geometry)	Geometry Function.
3	F (Fresnel)	Fresnel (для направленного света).
4	Diffuse Only	Только диффузная часть от направленного света.
5	Specular Only	Только спекулярная часть от направленного света.
6	Diffuse IBL	Диффузный IBL.
7	Specular IBL	Спекулярный IBL.

Режим	Отображаемая компонента	Пояснение
8	Fresnel IBL	Fresnel для IBL.
9	BRDF LUT	Визуализация двух каналов LUT.
10	Irradiance Map	Визуализация irradiance map (с Reinhard тональным отображением).

Для режимов 1-5 используется направленный свет как источник для вычислений. Для 6-10 – соответствующие IBL-компоненты.

2.4.9. Тональное отображение и гамма-коррекция (для режима 0)

Действие	Пояснение
<code>color = color / (color + 1.0);</code>	Простое тональное отображение (Reinhard) для сжатия HDR в LDR.
<code>color = pow(color, 1.0 / 2.2);</code>	Гамма-коррекция (преобразование в sRGB).

Для отладочных режимов гамма-коррекция также применяется, но без тонального отображения.

Связь с лекциями: Reinhard tonemap и гамма-коррекция – Лекция 2 (HDR и Tone Mapping).

3. SkyboxVS.hlsl – вершинный шейдер скайбокса

Назначение: преобразует вершины сферы скайбокса так, чтобы они всегда были на бесконечности (глубина = 1), и передаёт направление для сэмплинга кубической текстуры.

3.1. Константный буфер

Буфер	Регистр	Поля	Назначение
CameraBuffer	b0	matrix view – матрица вида matrix proj – матрица проекции	Передаётся из C++ (структура SkyboxConstants).

3.2. Входные и выходные данные

Структура	Элементы	Назначение
VS_INPUT	float3 pos : POSITION	Позиция вершины на сфере (единичный радиус).
VS_OUTPUT	float4 pos : SV_POSITION – позиция в clip-space float3 dir : TEXCOORD0 – направление для кубмапы	

3.3. Основная функция main

Шаг	Действие	Пояснение

Шаг	Действие	Пояснение
1	<pre>matrix viewNoTranslation = view; viewNoTranslation._41 = 0; viewNoTranslation._42 = 0; viewNoTranslation._43 = 0;</pre>	Убирает перенос из матрицы вида, чтобы скайбокс всегда был центрирован на камере.
2	<pre>float4 viewPos = mul(float4(input.pos, 1.0f), viewNoTranslation); float4 projPos = mul(viewPos, proj);</pre>	Преобразование в пространство вида и проекции (без переноса).
3	<pre>output.pos = float4(projPos.xy, projPos.w, projPos.w);</pre>	Устанавливает глубину равной w (т.е. 1.0 после перспективного деления) – скайбокс всегда на дальней плоскости.
4	<pre>output.dir = normalize(mul(input.pos, (float3x3)viewNoTranslation));</pre>	Направление для сэмплинга кубмапы (повёрнутое с учётом ориентации камеры, но без переноса).

Связь с лекциями: Стандартный приём для скайбокса (удаление переноса, фиксация глубины). Упоминается в Лекции 4 как часть IBL-окружения.

4. SkyboxPS.hlsl – пиксельный шейдер скайбокса

Назначение: сэмплирует кубическую текстуру (из IBL) по направлению и возвращает цвет с учётом ночного затухания.

4.1. Текстуры и сэмплеры

Регистр	Текстура	Назначение
t9	skyboxTexture	Кубическая текстура (из IBL).
s0	skyboxSampler	Сэмплер с линейной фильтрацией и clamp.

4.2. Входные данные

Структура	Элементы	Назначение
PS_INPUT	<pre>float4 pos : SV_POSITION – не используется float3 dir : TEXCOORD0 – направление из VS</pre>	

4.3. Основная функция main

Шаг	Действие	Пояснение
1	<pre>float3 dir = normalize(input.dir);</pre>	Нормализация направления.
2	<pre>float3 color = skyboxTexture.Sample(skyboxSampler, dir).rgb;</pre>	Сэмплинг кубмапы.
3	<pre>color *= 0.2f;</pre>	Ночное затухание (итерация 4).
4	<pre>return float4(color, 1.0f);</pre>	Вывод цвета (альфа = 1).

Связь с лекциями: Скайбокс как часть IBL-окружения, описан в Лекции 4. Затухание не является стандартной техникой, но добавлено для ночной сцены.

5. Сводная таблица по всем шейдерам

Шейдер	Вершинный шейдер	Пиксельный шейдер	Основные техники
Terrain	TerrainVS.hlsl	TerrainPS.hlsl	PBR (Cook-Torrance), IBL (диффузный + specular), Flow-эффект, направленный свет, точечные источники (закомментированы), отладочные режимы, тональное отображение (Reinhard), гамма-коррекция.
Skybox	SkyboxVS.hlsl	SkyboxPS.hlsl	Отрисовка сферы с кубической текстурой, удаление переноса, фиксация глубины, ночное затухание.

Заключение:

- Все шейдеры тесно связаны с теоретическим материалом лекций: PBR (Лекция 3), IBL (Лекции 4 и 5), постобработка/тонирование (Лекция 2).
- Реализованы современные техники: importance sampling, BRDF LUT, использование MIP-уровней для prefiltered env map.
- Flow-эффект – оригинальная разработка (не из лекций), но базируется на стандартных операциях с текстурами.
- Отладочные режимы – полезный инструмент для верификации.

Детальный разбор оставшихся .hlsl файлов и .bat файла

1. FullscreenQuad_vs.hlsl – вершинный шейдер для полноэкранного треугольника

Назначение: рисует три вершины, образующие треугольник, покрывающий весь экран (или текстуру). Используется при генерации Irradiance Map, Prefiltered Env Map и BRDF LUT. В отличие от HDRToCubeMap_vs, этот шейдер не вычисляет направление в мировом пространстве, а просто передаёт координаты.

1.1. Входные и выходные данные

Структура	Элементы	Назначение
VS_INPUT	uint vertexId : SV_VertexID	Индекс вершины (0, 1, 2).
PS_INPUT	float4 position : SV_POSITION – позиция в clip-space float4 localPos : POSITION1 – координаты вершины в диапазоне [-1,1] (для использования в пиксельном шейдере как UV или направление)	

1.2. Основная функция main

Шаг	Действие	Пояснение
1	Определяет массив из трёх позиций: float2 pos[3] = { {-1,-1}, {-1,3}, {3,-1} }	Треугольник, покрывающий экран (в clip-space). Это стандартный приём для рендеринга полноэкранных эффектов без вершинного буфера.
2	output.position = float4(pos[vertexId], 0.0, 1.0);	Передаёт позицию в clip-space.
3	output.localPos = float4(pos[vertexId].x, pos[vertexId].y, 1.0, 1.0);	Сохраняет координаты вершины (x,y) для передачи в PS. В BRDFLUT_PS эти координаты используются как UV (преобразуются в [0,1]). В других шейдерах они не используются напрямую (т.к. там используется направление из localDir).

Связь с лекциями: Этот шейдер используется во всех IBL-генерациях (Лекции 4 и 5). Сам по себе не несёт сложной логики, только обеспечивает покрытие экрана.

2. HDRToCubeMap_vs.hlsl – вершинный шейдер для генерации кубмапы из HDR

Назначение: для каждой грани кубмапы рисует полноэкранный квад, вычисляет направление в мировом пространстве из обратной матрицы view-projection и передаёт его в пиксельный шейдер для сэмплинга HDR-панорамы.

2.1. Константный буфер

Буфер	Регистр	Поля	Назначение
CubeMapCB	b0	float4x4 invViewProj	Обратная матрица view-projection для текущей грани.

2.2. Входные и выходные данные

Структура	Элементы	Назначение
-----------	----------	------------

Структура	Элементы	Назначение
PS_INPUT	float4 position : SV_POSITION – позиция в clip-space float3 localDir : TEXCOORD0 – направление в мировом пространстве	

2.3. Основная функция main

Шаг	Действие	Пояснение
1	Определяет массив из 4 позиций квада: float2 pos[4] = { {-1,1}, {1,1}, {-1,-1}, {1,-1} }	Полноэкранный квад (TRIANGLESTRIP).
2	float2 p = pos[vertexId]; o.position = float4(p, 0.0, 1.0);	Передаёт позицию.
3	float4 clip = float4(p, 1.0, 1.0); float4 world = mul(clip, invViewProj);	Преобразует координаты квада (в clip-space с глубиной 1) в мировое пространство с помощью обратной матрицы.
4	o.localDir = normalize(world.xyz / world.w);	Нормализует полученное направление. Это и будет направление луча, соответствующее данному пикселю грани кубмапы.

Связь с лекциями: Данный метод описан в Лекции 4 (преобразование equiangular → cubemap). Используется для рендеринга каждой грани кубмапы с соответствующей матрицей вида.

3. HDRToCubeMap_PS.hlsl – пиксельный шейдер для преобразования HDR в кубмапу

Назначение: сэмплирует 2D HDR-текстуру по сферическим координатам, полученным из направления, и возвращает цвет.

3.1. Текстуры и сэмплеры

Регистр	Текстура	Назначение
t0	tex	HDR-панорама (2D, формат float).
s0	smp1r	Линейный сэмплер (clamp).

3.2. Входные данные

Структура	Элементы	Назначение
PS_INPUT	float4 position : SV_POSITION – не используется float3 localDir : TEXCOORD0 – направление из VS	

3.3. Основная функция main

Шаг	Действие	Пояснение
1	float3 dir = normalize(input.localDir);	Нормализация направления.

Шаг	Действие	Пояснение
2	<code>float2 uv = float2(0.5 + atan2(dir.x, dir.z) / (2.0 * PI), 0.5 - asin(dir.y) / PI);</code>	Преобразование декартовых координат в сферические (equirectangular UV). – <code>atan2(dir.x, dir.z)</code> даёт азимутальный угол. – <code>asin(dir.y)</code> даёт угол места.
3	<code>uv.x = frac(uv.x);</code>	Обеспечивает горизонтальную цикличность (текстура заворачивается).
4	<code>uv.y = clamp(uv.y, 0.001, 0.999);</code>	Избегает проблем на полюсах (выход за границы текстуры).
5	<code>float4 color = tex.Sample(smplr, uv);</code>	Сэмплинг HDR-текстуры.
6	<code>return color;</code>	Возвращает цвет.

Связь с лекциями: Преобразование координат соответствует описанию в Лекции 4. Используется стандартная формула для equirectangular проекции.

4. IrradianceMap_PS.hlsl – пиксельный шейдер для генерации irradiance map

Назначение: для каждого направления (пикселя грани) вычисляет интеграл по полусфере от кубической карты, взвешенный по косинусу. Результат – irradiance (диффузный IBL).

4.1. Константы и буферы

Буфер/Константа	Назначение
PI	Число π.
<code>cbuffer IRRConstantBuffer : register(b0)</code>	Содержит <code>int4 params</code> – (Nphi, Ntheta, unused, unused). Количество семплов по азимуту и полярному углу.

4.2. Текстуры и сэмплеры

Регистр	Текстура	Назначение
t0	tex	Кубическая карта (исходная, из HDR).
s0	smplr	Сэмплер (линейный, без MIP).

4.3. Входные данные

Структура	Элементы	Назначение
PS_INPUT	<code>float4 position : SV_POSITION</code> – не используется <code>float3 localDir : TEXCOORD0</code> – направление из VS (нормаль для данной грани).	

4.4. Основная функция main

Шаг	Действие	Пояснение
-----	----------	-----------

Шаг	Действие	Пояснение
1	<code>float3 N = normalize(input.localDir);</code>	Нормаль.
2	Построение касательного базиса: <code>float3 up = abs(N.y) < 0.999 ? float3(0,1,0) : float3(1,0,0);</code> <code>float3 T = normalize(cross(up, N));</code> <code>float3 B = cross(N, T);</code>	Для интегрирования по полусфере требуется касательное пространство.
3	<code>int Nphi = max(params.x, 1); int Ntheta = max(params.y, 1);</code>	Количество семплов.
4	<code>float3 irradiance = float3(0,0,0);</code> <code>float dphi = 2.0 * PI / Nphi;</code> <code>float dtheta = (PI / 2.0) / Ntheta;</code>	Инициализация и шаги интегрирования (только верхняя полусфера, поэтому theta от 0 до PI/2).
5	Двойной цикл по i (phi) и j (theta): – Вычисляет <code>theta = (j + 0.5) * dtheta</code> (средняя точка). – Вычисляет <code>L_local</code> в касательном пространстве: <code>(sinTheta*cosPhi, sinTheta*sinPhi, cosTheta)</code> . – Преобразует в мировое: <code>L = normalize(L_local.x*T + L_local.y*B + L_local.z*N)</code> . – <code>NdotL = max(dot(N, L), 0.0)</code> . – Если <code>NdotL > 0</code> , сэмплирует <code>tex.SampleLevel(smplr, L, 0)</code> и добавляет вклад: <code>irradiance += radiance * NdotL * sinTheta * dtheta * dphi</code> .	Интегрирование методом Римана. Весовой множитель включает <code>sinTheta</code> (элемент телесного угла) и шаги.
6	<code>irradiance = irradiance / PI;</code>	Нормировка: для Ламбертова BRDF нужно поделить на π .
7	<code>return float4(irradiance, 1.0);</code>	

Связь с лекциями: Полностью соответствует описанию диффузного IBL из Лекции 4. Используется метод Римана с равномерной сеткой. Количество семплов (256x128) обеспечивает хорошее качество.

5. `PrefilterEnvMap_PS.hlsl` – пиксельный шейдер для `prefiltered environment map`

Назначение: для каждого направления и заданной шероховатости вычисляет свёртку кубической карты с использованием importance sampling GGX. Результат сохраняется в соответствующем MIP-уровне.

5.1. Константы и буферы

Буфер	Регистр	Поля	Назначение
PrefilConstantbuffer	b0	float4 roughness4	Содержит значение roughness для текущего MIP-уровня.

5.2. Текстуры и сэмплеры

Регистр	Текстура	Назначение
t0	tex	Кубическая карта (исходная).
s0	smplr	Сэмплер с линейной фильтрацией и поддержкой MIP (для выборки с mipLevel).

5.3. Вспомогательные функции

Функция	Назначение
sqr, posDot	Вспомогательные для возведения в квадрат и max(dot, 0).
RadicalInverse_VdC	Инверсия битов (последовательность Ван дер Корпута).
Hammersley	Генерирует пару (i/N, RadicalInverse) для последовательности Хаммерсли.
ImportanceSampleGGX	По заданным Xi (случайные) и roughness генерирует направление H (полусумма) с распределением GGX.
normalDistribution	Вычисляет NDF (GGX) для использования в PDF.

5.4. Основная функция main

Шаг	Действие	Пояснение
1	<pre>float3 norm = normalize(input.localDir); float3 view = norm;</pre>	Нормаль и направление взгляда (принято, что взгляд совпадает с нормалью для предварительной свёртки).
2	<pre>float roughness = roughness4.x; static const uint SAMPLE_COUNT = 256u;</pre>	Задано 256 семплов.
3	<pre>Цикл по i от 0 до SAMPLE_COUNT-1: - float2 Xi = Hammersley(i, SAMPLE_COUNT); - float3 H = ImportanceSampleGGX(Xi, norm, roughness); - float3 L = normalize(2.0 * dot(view, H) * H - view); (вычисление направления на источник). - float ndotl = max(dot(norm, L), 0.0); - float ndoth = max(dot(norm, H), 0.0); - float hdotv = max(dot(H, view), 0.0); - float D = normalDistribution(H, norm, roughness); - float pdf = (D * ndoth / (4.0 * hdotv)) + 0.0001; (плотность вероятности). - Вычисление mipLevel на основе разрешения и pdf: float resolution = 256.0; float saTexel = 4.0 * PI / (6.0 * resolution * resolution); float saSample = 1.0 / (SAMPLE_COUNT * pdf + 0.0001); float mipLevel = roughness == 0.0 ? 0.0 : 0.5 * log2(saSample / saTexel); - Если ndotl > 0, добавляет вклад: prefilteredColor += tex.SampleLevel(smplr, L, mipLevel) * ndotl; - totalWeight += ndotl;</pre>	Importance sampling с весами по ndotl. Выборка с MIP-уровнем для сглаживания при высоких roughness.
4	<pre>prefilteredColor = prefilteredColor / totalWeight;</pre>	Нормировка.
5	<pre>return float4(prefilteredColor, 1.0f);</pre>	

Связь с лекциями: Полностью соответствует описанию спекулярного IBL из Лекции 5. Используется последовательность Хаммерсли, importance sampling GGX и вычисление mip-уровня для фильтрации. Количество семплов 256 – хороший компромисс между качеством и производительностью.

6. BRDFLUT_PS.hlsl – пиксельный шейдер для генерации BRDF LUT

Назначение: для каждой пары (NdotV, roughness) вычисляет два коэффициента (scale и bias) путём интегрирования BRDF с использованием importance sampling. Результат – 2D-текстура, используемая в основном шейдере для аппроксимации интеграла спекулярного IBL.

6.1. Структура и константы

Константа	Значение	Назначение
SAMPLE_COUNT	1024u	Количество семплов для интегрирования.
PI	3.1415926	Число π.

6.2. Вспомогательные функции

Функция	Назначение
sqr, posDot	Вспомогательные.
SchlickGGX	Geometry (Schlick-GGX) для одного направления.
geometry	Smith Geometry (произведение двух).
RadicalInverse_VdC, Hammersley, ImportanceSampleGGX	Те же, что и в PrefilterEnvMap_PS.

6.3. Основная функция *IntegrateBRDF*

Аргументы	Назначение	
float NdotV	Косинус угла между нормалью и взглядом (от 0 до 1).	
float roughness	Шероховатость (от 0 до 1).	
Шаг	Действие	Пояснение
1	float3 V; V.x = sqrt(1.0 - NdotV * NdotV); V.z = 0.0; V.y = NdotV;	Строит вектор взгляда в касательном пространстве (N = (0,1,0)).
2	float A = 0.0, B = 0.0; float3 N = float3(0.0, 1.0, 0.0);	Инициализация коэффициентов.
3	Цикл по i от 0 до SAMPLE_COUNT-1: – Генерирует Xi = Hammersley(i, SAMPLE_COUNT). – H = ImportanceSampleGGX(Xi, N, roughness). – L = normalize(2.0 * dot(V, H) * H - V). – NdotL = max(L.y, 0.0).	Интегрирование: – G_Vis – это (G * VdotH) / (NdotH * NdotV). – (1 - Fc) и Fc – разложение Френеля на две части. – A накапливает коэффициент при F0, B – при (1-F0).

Аргументы	Назначение	
	–NdotH = max(H.y, 0.0). –VdotH = max(dot(V, H), 0.0). –Если NdotL > 0: float G = geometry(N, V, L, roughness); float G_Vis = (G * VdotH) / max(NdotH * NdotV, 0.0001f); float Fc = pow(1.0 - VdotH, 5.0); A += (1.0 - Fc) * G_Vis; B += Fc * G_Vis;	
4	A /= SAMPLE_COUNT; B /= SAMPLE_COUNT;	Усреднение.
5	return float2(A, B);	Возвращает scale и bias.

6.4. Основная функция *main* (точка входа)

Шаг	Действие	Пояснение
1	float2 outpos = (float2(input.localPos.x, input.localPos.y) + 1) / 2;	Преобразует координаты вершины из [-1,1] в [0,1] (UV).
2	float NdotV = outpos.x; float r = outpos.y;	X – NdotV, Y – roughness.
3	return IntegrateBRDF(NdotV, r);	Вызов интегрирования.

Связь с лекциями: Полностью соответствует алгоритму генерации BRDF LUT из Лекции 5. Используется importance sampling с GGX, разложение Френеля и накопление двух коэффициентов. Результат используется в основном шейдере для аппроксимации интеграла specular IBL.

7.compile_shaders.bat – пакетный файл для компиляции шейдеров

Назначение: автоматизирует компиляцию всех HLSL-шейдеров в CSO-файлы с помощью утилиты fxc.exe из Windows SDK.

7.1. Содержимое

```
batch

@echo off
set FXC="C:\Program Files (x86)\Windows Kits\10\bin\10.0.17763.0\x64\fxc.exe"

%FXC% /T vs_5_0 /E main /Fo FullscreenQuad_VS.cso FullscreenQuad_VS.hlsl
%FXC% /T vs_5_0 /E main /Fo HDRToCubeMap_VS.cso HDRToCubeMap_VS.hlsl
%FXC% /T ps_5_0 /E main /Fo HDRToCubeMap_PS.cso HDRToCubeMap_PS.hlsl
%FXC% /T ps_5_0 /E main /Fo IrradianceMap_PS.cso IrradianceMap_PS.hlsl
%FXC% /T ps_5_0 /E main /Fo PrefilterEnvMap_PS.cso PrefilterEnvMap_PS.hlsl
%FXC% /T ps_5_0 /E main /Fo BRDFLUT_PS.cso BRDFLUT_PS.hlsl

echo All shaders compiled to .cso
pause
```

7.2. Пояснение параметров fxc

Параметр	Назначение
/T vs_5_0 или /T ps_5_0	Целевой профиль шейдера (вершинный или пиксельный, версия 5.0).
/E main	Точка входа – функция main.
/Fo output.cso	Выходной файл (скомпилированный шейдер в формате CSO).
input.hlsl	Исходный файл.

Связь с лекциями: Компиляция шейдеров упоминается в Лекции 1 (диагностика) – использование fxc и отладочных флагов. В данном .bat-файле не заданы флаги отладки, но в коде C++ при компиляции через D3DCompileFromFile используются флаги DEBUG и SKIP_OPTIMIZATION для отладки.

8. Сводная таблица всех HLSL-файлов (IBL-генерация)

Файл	Тип	Назначение	Входные текстуры	Выходные текстуры	Ключевые техники	Связь с лекцией
FullscreenQuad_VS.hlsl	VS	Полноэкранный треугольник	Нет	–	–	–
HDRToCubeMap_VS.hlsl	VS	Вычисление направления для кубмапы	Нет	–	Обратная матрица view-proj, перспективное деление	Лекция 4
HDRToCubeMap_PS.hlsl	PS	Преобразование equirectangular → cubemap	HDR-панорама (2D)	Кубическая карта	Преобразование координат (atan2, asin)	Лекция 4
IrradianceMap_PS.hlsl	PS	Генерация irradiance map (диффузный IBL)	Кубическая карта	Irradiance map	Интегрирование по полусфере, метод Римана, нормировка на π	Лекция 4
PrefilterEnvMap_PS.hlsl	PS	Генерация prefILTERed env map (спекулярный IBL)	Кубическая карта	Prefiltered map (с MIP)	Importance sampling GGX, последовательность Хаммерсли, вычисление mipLevel	Лекция 5
BRDFLUT_PS.hlsl	PS	Генерация BRDF LUT	Нет	2D LUT (R16G16_FLOAT)	Интегрирование BRDF, разложение Френеля, importance sampling	Лекция 5

9. Заключение

- **Все IBL-шейдеры** строго следуют теоретическому материалу лекций 4 и 5.
- Используются передовые техники: importance sampling, последовательности Хаммерсли, адаптивная выборка с MIP-уровнями.
- compile_shaders.bat обеспечивает простую компиляцию всех шейдеров, что удобно для разработки.
- Код шейдеров хорошо структурирован и содержит комментарии (в основном, на русском), что облегчает понимание.

Таким образом, весь проект является качественной реализацией современных алгоритмов рендеринга, демонстрирующей глубокое понимание PBR и IBL.